Chapter 14: Exception Handling & Robust Debugging

Creating Custom Exceptions, Handling Multiple Errors, and Exception Chaining

Source: Python Cookbook, 3rd Edition (David Beazley & Brian K. Jones) | Week 1 Reference

---

Table of Contents & Core Recipes

• **Recipe 14.6: Handling Multiple Exceptions** — Catching related error types cleanly in single or tiered except blocks

• **Recipe 14.7: Catching All Exceptions** — When and how to catch Exception safely without swallowing system signals

• **Recipe 14.8: Creating Custom Exceptions** — Defining custom exception classes inheriting from Exception or domain bases

• **Recipe 14.9: Raising an Exception in Response to Another** — Exception chaining using 'raise ... from err' to preserve tracebacks

• **Recipe 14.10: Reraising the Last Exception** — Using naked 'raise' inside an except block after logging

```
    @unittest.skipUnless(platform.system() == 'Darwin', 'Mac specific test')
    def test_3(self):
        self.assertTrue(True)

    @unittest.expectedFailure
    def test_4(self):
        self.assertEqual(2+2, 5)

if __name__ == '__main__':
    unittest.main()
```

If you run this code on a Mac, you'll get this output:

```
bash % python3 testsample.py -v
test_0 (__main__.Tests) ... ok
test_1 (__main__.Tests) ... skipped 'skipped test'
test_2 (__main__.Tests) ... skipped 'Not supported on Unix'
test_3 (__main__.Tests) ... ok
test_4 (__main__.Tests) ... expected failure

----------------------------------------------------------------------
Ran 5 tests in 0.002s

OK (skipped=2, expected failures=1)
```

Discussion

The `skip()` decorator can be used to skip over a test that you don't want to run at all. `skipIf()` and `skipUnless()` can be a useful way to write tests that only apply to certain platforms or Python versions, or which have other dependencies. Use the `@expectedFailure` decorator to mark tests that are known failures, but for which you don't want the test framework to report more information.

The decorators for skipping methods can also be applied to entire testing classes. For example:

```
@unittest.skipUnless(platform.system() == 'Darwin', 'Mac specific tests')
class DarwinTests(unittest.TestCase):
    ...
```

14.6. Handling Multiple Exceptions

Problem

You have a piece of code that can throw any of several different exceptions, and you need to account for all of the potential exceptions that could be raised without creating duplicate code or long, meandering code passages.

Solution

If you can handle different exceptions all using a single block of code, they can be grouped together in a tuple like this:

```
try:
    client_obj.get_url(url)
except (URLError, ValueError, SocketTimeout):
    client_obj.remove_url(url)
```

In the preceding example, the `remove_url()` method will be called if any one of the listed exceptions occurs. If, on the other hand, you need to handle one of the exceptions differently, put it into its own `except` clause:

```
try:
    client_obj.get_url(url)
except (URLError, ValueError):
    client_obj.remove_url(url)
except SocketTimeout:
    client_obj.handle_url_timeout(url)
```

Many exceptions are grouped into an inheritance hierarchy. For such exceptions, you can catch all of them by simply specifying a base class. For example, instead of writing code like this:

```
try:
    f = open(filename)
except (FileNotFoundError, PermissionError):
    ...
```

you could rewrite the `except` statement as:

```
try:
    f = open(filename)
except OSError:
    ...
```

This works because `OSError` is a base class that's common to both the `FileNotFoundError`and `PermissionError` exceptions.

Discussion

Although it's not specific to handling *multiple* exceptions per se, it's worth noting that you can get a handle to the thrown exception using the `as` keyword:

```
try:
    f = open(filename)
except OSError as e:
    if e.errno == errno.ENOENT:
        logger.error('File not found')
    elif e.errno == errno.EACCES:
        logger.error('Permission denied')
```

```
        else:
            logger.error('Unexpected error: %d', e.errno)
```

In this example, the `e` variable holds an instance of the raised `OSError`. This is useful if you need to inspect the exception further, such as processing it based on the value of an additional status code.

Be aware that `except` clauses are checked in the order listed and that the first match executes. It may be a bit pathological, but you can easily create situations where multiple `except` clauses might match. For example:

```
>>> f = open('missing')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
FileNotFoundError: [Errno 2] No such file or directory: 'missing'
>>> try:
...     f = open('missing')
... except OSError:
...     print('It failed')
... except FileNotFoundError:
...     print('File not found')
...
It failed
>>>
```

Here the `except FileNotFoundError` clause doesn't execute because the `OSError` is more general, matches the `FileNotFoundError` exception, and was listed first.

As a debugging tip, if you're not entirely sure about the class hierarchy of a particular exception, you can quickly view it by inspecting the exception's `__mro__` attribute. For example:

```
>>> FileNotFoundError.__mro__
(<class 'FileNotFoundError'>, <class 'OSError'>, <class 'Exception'>,
 <class 'BaseException'>, <class 'object'>)
>>>
```

Any one of the listed classes up to `BaseException` can be used with the `except` statement.

14.7. Catching All Exceptions

Problem

You want to write code that catches all exceptions.

Solution

To catch all exceptions, write an exception handler for `Exception`, as shown here:

```
try:
    ...
except Exception as e:
    ...
    log('Reason:', e)       # Important!
```

This will catch all exceptions save `SystemExit`, `KeyboardInterrupt`, and `GeneratorExit`. If you also want to catch those exceptions, change `Exception` to `BaseException`.

Discussion

Catching all exceptions is sometimes used as a crutch by programmers who can't remember all of the possible exceptions that might occur in complicated operations. As such, it is also a very good way to write undebuggable code if you are not careful.

Because of this, if you choose to catch all exceptions, it is absolutely critical to log or report the actual reason for the exception somewhere (e.g., log file, error message printed to screen, etc.). If you don't do this, your head will likely explode at some point. Consider this example:

```
def parse_int(s):
    try:
        n = int(v)
    except Exception:
        print("Couldn't parse")
```

If you try this function, it behaves like this:

```
>>> parse_int('n/a')
Couldn't parse
>>> parse_int('42')
Couldn't parse
>>>
```

At this point, you might be left scratching your head as to why it doesn't work. Now suppose the function had been written like this:

```
def parse_int(s):
    try:
        n = int(v)
    except Exception as e:
        print("Couldn't parse")
        print('Reason:', e)
```

In this case, you get the following output, which indicates that a programming mistake has been made:

```
>>> parse_int('42')
Couldn't parse
Reason: global name 'v' is not defined
>>>
```

All things being equal, it's probably better to be as precise as possible in your exception handling. However, if you must catch all exceptions, just make sure you give good diagnostic information or propagate the exception so that cause doesn't get lost.

14.8. Creating Custom Exceptions

Problem

You're building an application and would like to wrap lower-level exceptions with custom ones that have more meaning in the context of your application.

Solution

Creating new exceptions is easy—just define them as classes that inherit from `Exception` (or one of the other existing exception types if it makes more sense). For example, if you are writing code related to network programming, you might define some custom exceptions like this:

```
class NetworkError(Exception):
    pass

class HostnameError(NetworkError):
    pass

class TimeoutError(NetworkError):
    pass

class ProtocolError(NetworkError):
    pass
```

Users could then use these exceptions in the normal way. For example:

```
try:
    msg = s.recv()
except TimeoutError as e:
    ...
except ProtocolError as e:
    ...
```

Discussion

Custom exception classes should almost always inherit from the built-in `Exception` class, or inherit from some locally defined base exception that itself inherits from `Exception`. Although all exceptions also derive from `BaseException`, you should not use this as a base class for new exceptions. `BaseException` is reserved for system-exiting exceptions, such as `KeyboardInterrupt` or `SystemExit`, and other exceptions that should signal the application to exit. Therefore, catching these exceptions is not the

intended use case. Assuming you follow this convention, it follows that inheriting from `BaseException` causes your custom exceptions to not be caught and to signal an imminent application shutdown!

Having custom exceptions in your application and using them as shown makes your application code tell a more coherent story to whoever may need to read the code. One design consideration involves the grouping of custom exceptions via inheritance. In complicated applications, it may make sense to introduce further base classes that group different classes of exceptions together. This gives the user a choice of catching a narrowly specified error, such as this:

```
try:
    s.send(msg)
except ProtocolError:
    ...
```

It also gives the ability to catch a broad range of errors, such as the following:

```
try:
    s.send(msg)
except NetworkError:
    ...
```

If you are going to define a new exception that overrides the `__init__()` method of `Exception`, make sure you always call `Exception.__init__()` with all of the passed arguments. For example:

```
class CustomError(Exception):
    def __init__(self, message, status):
        super().__init__(message, status)
        self.message = message
        self.status = status
```

This might look a little weird, but the default behavior of `Exception` is to accept all arguments passed and to store them in the `.args` attribute as a tuple. Various other libraries and parts of Python expect all exceptions to have the `.args` attribute, so if you skip this step, you might find that your new exception doesn't behave quite right in certain contexts. To illustrate the use of `.args`, consider this interactive session with the built-in `RuntimeError` exception, and notice how any number of arguments can be used with the `raise` statement:

```
>>> try:
...     raise RuntimeError('It failed')
... except RuntimeError as e:
...     print(e.args)
...
('It failed',)
>>> try:
...     raise RuntimeError('It failed', 42, 'spam')
... except RuntimeError as e:
```

```
...     print(e.args)
...
('It failed', 42, 'spam')
>>>
```

For more information on creating your own exceptions, see the Python documentation.

14.9. Raising an Exception in Response to Another Exception

Problem

You want to raise an exception in response to catching a different exception, but want to include information about both exceptions in the traceback.

Solution

To chain exceptions, use the `raise from` statement instead of a simple `raise` statement. This will give you information about both errors. For example:

```
>>> def example():
...     try:
...             int('N/A')
...     except ValueError as e:
...             raise RuntimeError('A parsing error occurred') from e...
>>>
example()
Traceback (most recent call last):
  File "<stdin>", line 3, in example
ValueError: invalid literal for int() with base 10: 'N/A'

The above exception was the direct cause of the following exception:

Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 5, in example
RuntimeError: A parsing error occurred
>>>
```

As you can see in the traceback, both exceptions are captured. To catch such an exception, you would use a normal `except` statement. However, you can look at the `__cause__` attribute of the exception object to follow the exception chain should you wish. For example:

```
try:
    example()
except RuntimeError as e:
    print("It didn't work:", e)
```

```
    if e.__cause__:
        print('Cause:', e.__cause__)
```

An implicit form of chained exceptions occurs when another exception gets raised inside an `except` block. For example:

```
>>> def example2():
...     try:
...             int('N/A')
...     except ValueError as e:
...             print("Couldn't parse:", err)
...
>>>
>>> example2()
Traceback (most recent call last):
  File "<stdin>", line 3, in example2
ValueError: invalid literal for int() with base 10: 'N/A'

During handling of the above exception, another exception occurred:

Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 5, in example2
NameError: global name 'err' is not defined
>>>
```

In this example, you get information about both exceptions, but the interpretation is a bit different. In this case, the `NameError` exception is raised as the result of a programming error, not in direct response to the parsing error. For this case, the `__cause__` attribute of an exception is not set. Instead, a `__context__` attribute is set to the prior exception.

If, for some reason, you want to suppress chaining, use `raise from None`:

```
>>> def example3():
...     try:
...             int('N/A')
...     except ValueError:
...             raise RuntimeError('A parsing error occurred') from None...
>>>
example3()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 5, in example3
RuntimeError: A parsing error occurred
>>>
```

Discussion

In designing code, you should give careful attention to use of the `raise` statement inside of other `except` blocks. In most cases, such `raise` statements should probably be changed to `raise from` statements. That is, you should prefer this style:

```
try:
   ...
except SomeException as e:
   raise DifferentException() from e
```

The reason for doing this is that you are explicitly chaining the causes together. That is, the `DifferentException` is being raised in direct response to getting a `SomeException`. This relationship will be explicitly stated in the resulting traceback.

If you write your code in the following style, you still get a chained exception, but it's often not clear if the exception chain was intentional or the result of an unforeseen programming error:

```
try:
   ...
except SomeException:
   raise DifferentException()
```

When you use `raise from`, you're making it clear that you meant to raise the second exception.

Resist the urge to suppress exception information, as shown in the last example. Although suppressing exception information can lead to smaller tracebacks, it also discards information that might be useful for debugging. All things being equal, it's often best to keep as much information as possible.

14.10. Reraising the Last Exception

Problem

You caught an exception in an `except` block, but now you want to reraise it.

Solution

Simply use the `raise` statement all by itself. For example:

```
>>> def example():
...     try:
...             int('N/A')
...     except ValueError:
...             print("Didn't work")
...             raise
...
```

```
>>> example()
Didn't work
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 3, in example
ValueError: invalid literal for int() with base 10: 'N/A'
>>>
```

Discussion

This problem typically arises when you need to take some kind of action in response to an exception (e.g., logging, cleanup, etc.), but afterward, you simply want to propagate the exception along. A very common use might be in catch-all exception handlers:

```
try:
   ...
except Exception as e:
   # Process exception information in some way
   ...

   # Propagate the exception
   raise
```

14.11. Issuing Warning Messages

Problem

You want to have your program issue warning messages (e.g., about deprecated features or usage problems).

Solution

To have your program issue a warning message, use the `warnings.warn()` function. For example:

```
import warnings

def func(x, y, logfile=None, debug=False):
    if logfile is not None:
         warnings.warn('logfile argument deprecated', DeprecationWarning)
    ...
```

The arguments to `warn()` are a warning message along with a warning class, which is typically one of the following: `UserWarning`, `DeprecationWarning`, `SyntaxWarning`, `RuntimeWarning`, `ResourceWarning`, or `FutureWarning`.

The handling of warnings depends on how you have executed the interpreter and other configuration. For example, if you run Python with the `-W all` option, you'll get output such as the following:

```
bash % python3 -W all example.py
example.py:5: DeprecationWarning: logfile argument is deprecated
  warnings.warn('logfile argument is deprecated', DeprecationWarning)
```

Normally, warnings just produce output messages on standard error. If you want to turn warnings into exceptions, use the `-W error` option:

```
bash % python3 -W error example.py
Traceback (most recent call last):
  File "example.py", line 10, in <module>
    func(2, 3, logfile='log.txt')
  File "example.py", line 5, in func
    warnings.warn('logfile argument is deprecated', DeprecationWarning)
DeprecationWarning: logfile argument is deprecated
bash %
```

Discussion

Issuing a warning message is often a useful technique for maintaining software and assisting users with issues that don't necessarily rise to the level of being a full-fledged exception. For example, if you're going to change the behavior of a library or framework, you can start issuing warning messages for the parts that you're going to change while still providing backward compatibility for a time. You can also warn users about problematic usage issues in their code.

As another example of a warning in the built-in library, here is an example of a warning message generated by destroying a file without closing it:

```
>>> import warnings
>>> warnings.simplefilter('always')
>>> f = open('/etc/passwd')
>>> del f
__main__:1: ResourceWarning: unclosed file <_io.TextIOWrapper name='/etc/passwd'
 mode='r' encoding='UTF-8'>
>>>
```

By default, not all warning messages appear. The `-W` option to Python can control the output of warning messages. `-W all` will output all warning messages, `-W ignore` ignores all warnings, and `-W error` turns warnings into exceptions. As an alternative, you can can use the `warnings.simplefilter()` function to control output, as just shown. An argument of `always` makes all warning messages appear, `ignore` ignores all warnings, and `error` turns warnings into exceptions.

For simple cases, this is all you really need to issue warning messages. The `warnings` module provides a variety of more advanced configuration options related to the filtering and handling of warning messages. See the Python documentation for more information.